University of Essex
School of Computer Science and Electronic Engineering

CE264 Digital Systems Design – Spring Term 2021
Northwest University China
Tutorial on Multisim + Xilinx Vivado + Digilent Basys3

This document provides an initial tutorial for the first laboratory session of the term, to get all students started with the computer-aided design software that we will be using during the term. **You are expected to complete this tutorial to the point of a working circuit in hardware within your first two-hour lab session.**

In completing this tutorial you will learn how to use Multisim to enter a simple design schematic, and how to build that design and test it on a hardware board. The design is trivial – a few components only. The point is the **process** by which a design is converted into hardware, because until you understand that process, you cannot work on a design of your own. Engineers in industry normally do something very similar when learning to use new hardware or software for the first time.

Background

CE264 is about **design**. You cannot learn design from a book (in my view). You have to **learn by doing** design, and CE264 is a laboratory-based module in which you will learn by designing a non-trivial, but not too complicated, digital system. This system will be designed using schematics (circuit diagrams), and realised/implemented for a specific hardware circuit board called the Basys3, which contains a Xilinx Field-Programmable Gate Array (FPGA).

The FPGA is a chip with a large amount of logic and a modest amount of memory, that can be configured by loading it with data (bits) from a computer over a serial link interfaced to a USB socket on the Basys3 board.

FPGAs are widely used in industry, especially in systems which are manufactured in numbers ranging from tens to thousands (but not millions, where custom chips are cheaper). A significant advantage of FPGAs for education is that we can realise a circuit with hundreds of gates without having to wire it on the bench – your circuit can be translated from gate and component symbols on a schematic, to configuration bits that can be downloaded to the board. You can perform an edit, build, test cycle in a matter of minutes and see whether your circuit works on the board. This is educationally far superior to working with individual gates and wires as you may have done in Year 2, and also much closer to what digital designers do in industry, except that they no longer use schematics (industrial designs are coded in hardware description languages and synthesised into hardware – but we don't study it at Essex until final year module CE339 - High Level Digital Design).

The hardware boards you will use are Digilent Basys3 boards (full details are available on Moodle). This is our fifth year using Basys3 boards, and before that we used Basys2 boards for 5 years or so, so we're fairly familiar with the board. We are using Multisim 14 for schematic entry (and simulation if you use it). This is our fourth year using Multisim 14.

Once a circuit is ready to be implemented, it must be exported from Multisim in the form of VHDL files (which will be explained later), and imported into the Xilinx Vivado Design Suite for implementation onto the FPGA. This is a complex process that takes several steps, but from the

user's point of view, it is fairly straightforward.

Xilinx Vivado can then download the completed design to the Basys3 board, over a USB cable, and you can see whether your design works. To make this process easier, the design you will implement will have easily visible outputs (testing and verifying a more complex design is not so simple, but we can leave that to the final year study).

If you have not used Multisim, then you will need to consult the following tutorial, or something similar:

Introduction to Multisim: Learn to Capture, Simulate, and Layout in Less Than 30 Minutes
National Instruments, September 2014. http://www.ni.com/white-paper/10710/en/

Tutorial

You should work through the following tutorial in one laboratory session if possible and if you don't complete it, you should do so during the next session at most, otherwise in your own time. Make sure you understand what you are doing: completing this tutorial without understanding the steps will result in you wasting a lot of time in the following laboratory sessions.

Not every step is detailed in this document, it is assumed that you have some familiarity with using software, and a mouse, and are capable of guessing some steps from your previous experience. If you can't work out what to do, **ask!**

Make notes as needed to supplement what is in this document. Your logbook is the correct place for these notes.

Be aware that you need to work **accurately** in this sort of work. Sloppy editing and careless clicking with a mouse can cause many problems including corruption of your files. **So don't go too fast — learn to work accurately.**

The steps covered in this tutorial are:

1. Start the National Instruments Multisim software.
2. Create a *project* (a folder in which your designs and simulations are stored).
3. Create a schematic for a simple combinational circuit (given later).
4. Export the circuit to VHDL files.
5. Start the Xilinx Vivado software.
6. Create a *project* (this time a Vivado project, of course).
7. Import the VHDL files from step 4 and a Xilinx constraint file (explained later).
8. Synthesise logic from the VHDL files.
9. Implement the synthesised logic for the FPGA on the Basys3 board.
10. Download the implemented logic to the Basys3 board and see whether it works correctly.

Once you have been through this process **correctly** once, making changes to the circuit is much quicker – you only have to repeat step 4 and steps 8, 9, 10.

Follow the instructions carefully. Be warned that Xilinx Vivado, like most Computer-Aided Design (CAD) software, is designed for professionals who know what they are doing. It is complex

software, is not trivial to learn, and it has many capabilities and features that you will not need to use. Stick to the processes outlined here, and do not tamper with settings other than those that are described here. Also, do not interfere with the files created by the Xilinx software unless you are told to do so.

A warning about filenames: you will be able to choose filenames for your projects and schematics. You are advised not to use spaces in filenames (they may be OK, but it isn't worth the risk). Choose meaningful filenames and write them down in your logbook, so that you can work out what is in each file. In my experience many of the problems encountered by CE264 students are due to poor file management.

File Management

Before starting any software, create a new folder for your CE264 files ON YOUR M: DRIVE. Keep all your project files in this folder by making sure that the Multisim and Vivado software is using your M: drive and not the default, which is the C: drive on the machine. Files saved on the C: drive will not be backed up, and are not available on other machines, whereas files on your M: drive are available on any campus machine, and you can transfer them to your own personal machine if you wish. Do not work from files on a USB drive – this caused problems in a previous year, and USB drives are slower in any case, so are not a good idea. (It's OK to use a USB drive to keep backups.)

Use **separate** project folders within CE264 for this tutorial and for your later design work. Write down the names of these folders and what they are for. The folders will inevitably become filled with files created by the software. Do not delete any of these unless you really know what you are doing.

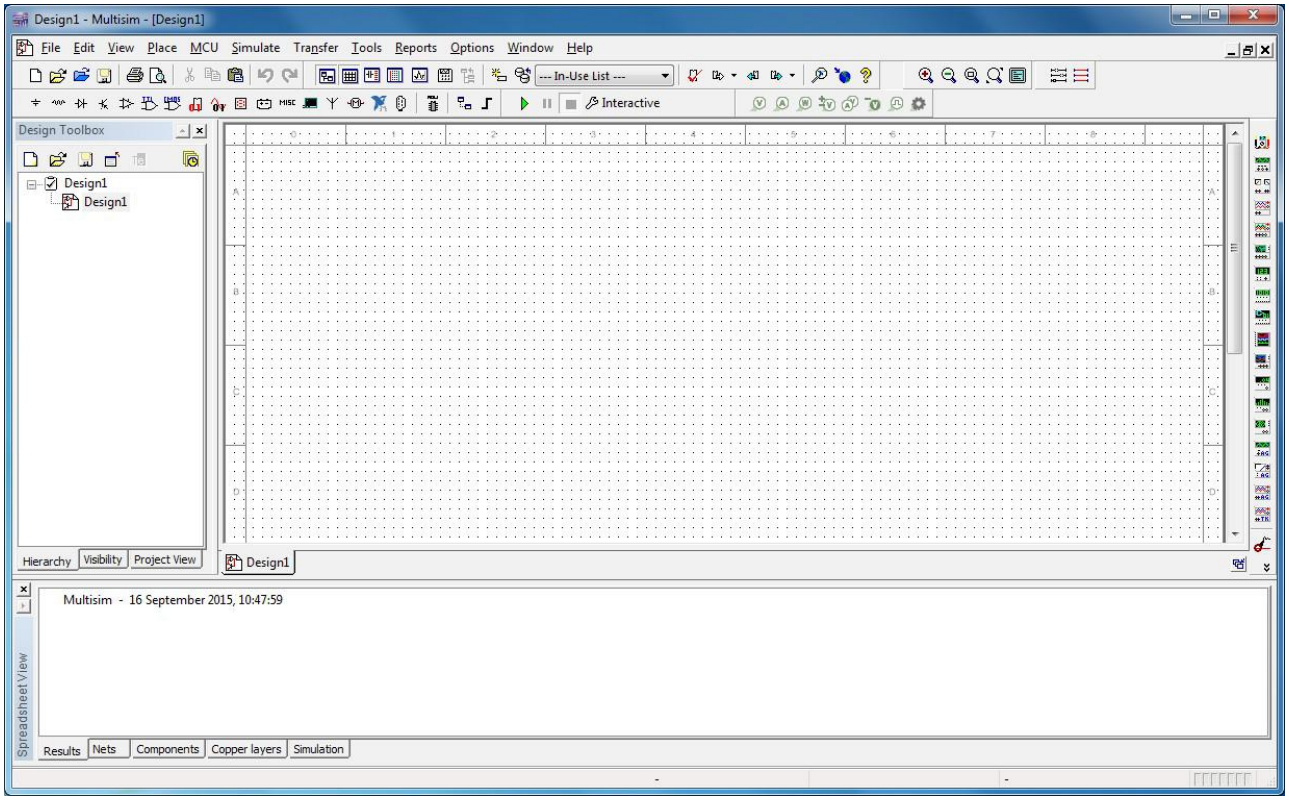
Hint:

Add the Multisim and Vivado programs to your Windows Start tiles, by drag and drop. This will save you time in every lab sesssion compared to scrolling down the Start menu list.

The screenshots and window captures that follow were obviously captured with an earlier version of Windows. However, they show what you need for this tutorial, and they have not been updated because of the time it takes to run the software and capture each window.

Start the Multisim 14 software

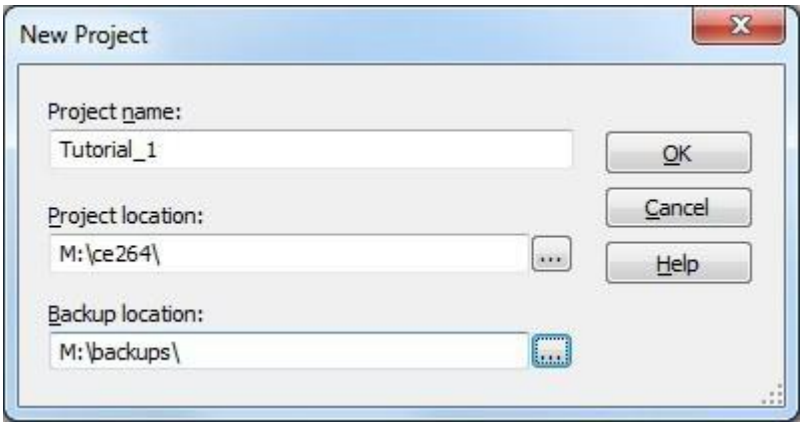
From the Windows Start menu, select: **NI Multisim 14.1**
and a splash screen should appear briefly, followed shortly after by the initial window shown below.



Create a project

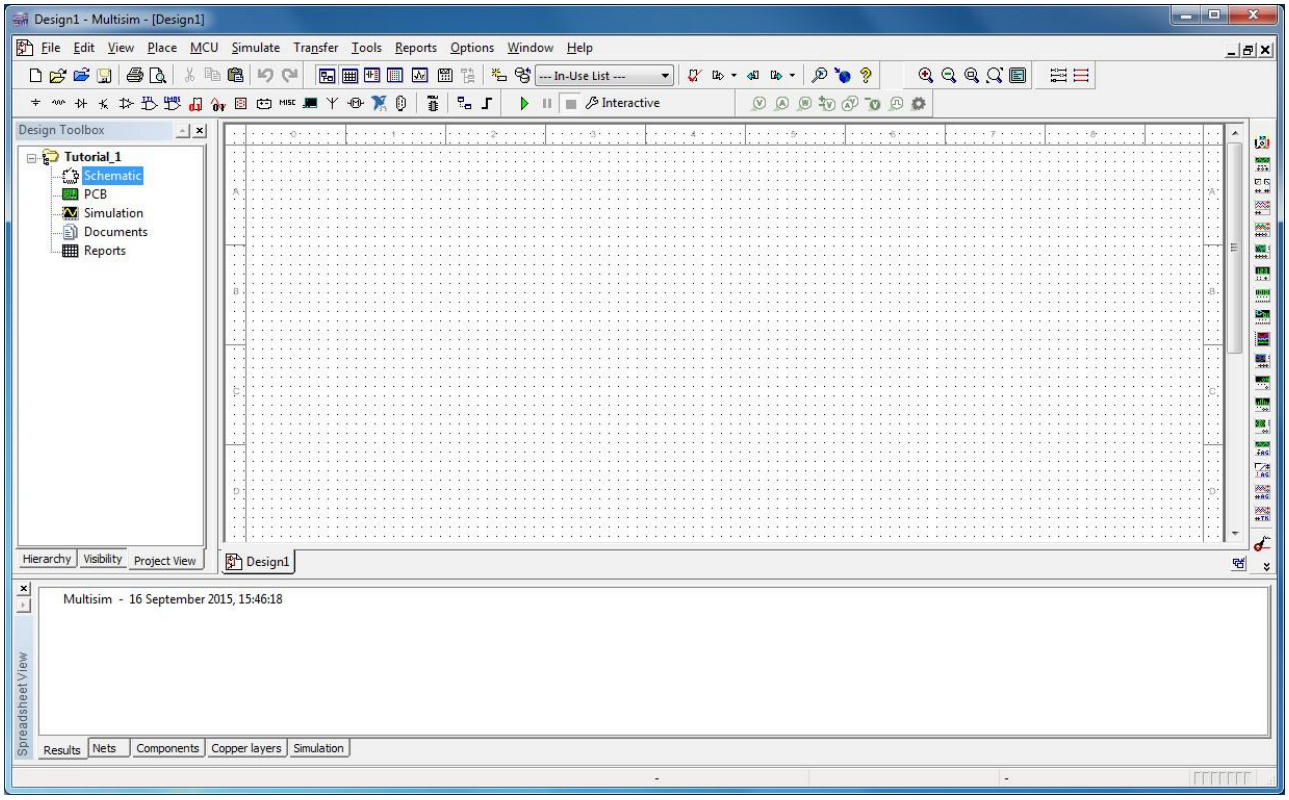
Before you can enter any designs, you must create a Multisim *project* – a database of files associated together as part of one design or family of designs. (This is how most CAD software works). You will create a project for this tutorial, and later create a new project for your design work as part of CE264. Do not mix the two projects together. Equally, do not create more projects – it is important to gather together all the parts of a design in a single project.

The project name could be tutorial_1 or something else of your choice. To create the project select **Projects and Packing → New project** from the **File** menu. A pop-up window like this will appear:



Multisim enforces a difference between the location of the project (M:\ce264\ in my case) and the location of the backup (M:\backups\ in my case). You may prefer to use something like M:\backups\ce264.

Once you have created the project, the main window should look like this. Notice the project structure shown in the Design Toolbox pane on the left.

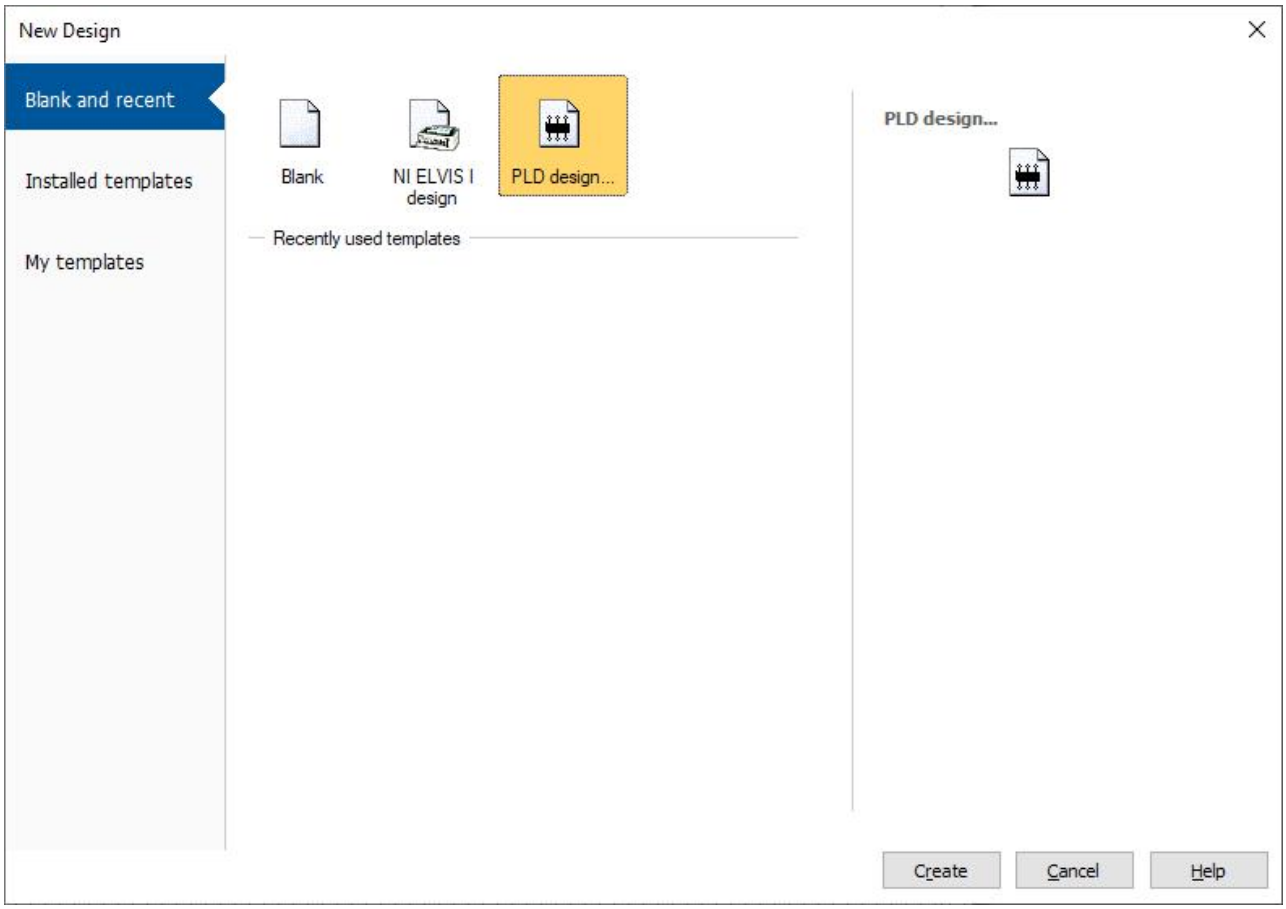


You are now ready to create a schematic. This project will need only one schematic (this won't be true of your next project, but we'll deal with that later). When you create a schematic, you need to tell Multisim that it will be a design for an FPGA. Multisim calls this a PLD design (PLD stands for Programmable Logic Device).

To create the schematic **within your project** click in the Design Toolbox pane, on the Schematic icon or text, and select → Add new file.

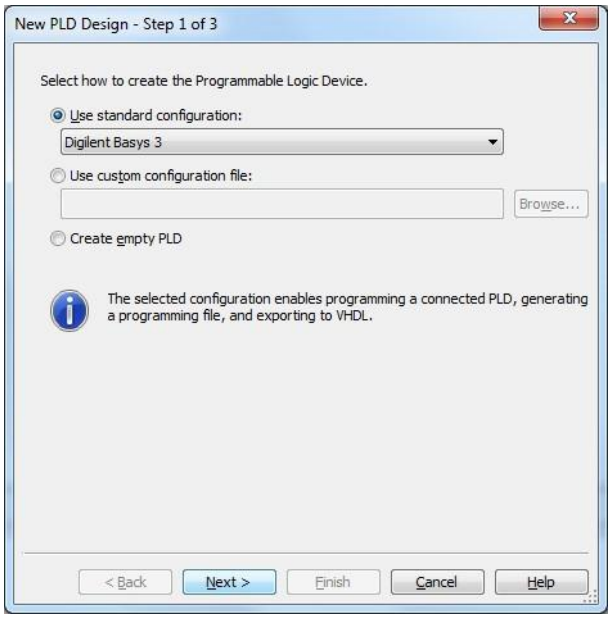
(It is also possible to create a schematic using **File → New → PLD design**, but in this case the created schematic will not be part of your project. Having a collection of independent schematics will make life very difficult for you, so don't fall into this trap. Later you will learn about hierarchy and its importance in design, and organizing schematics as part of a hierarchy within a project is absolutely essential.)

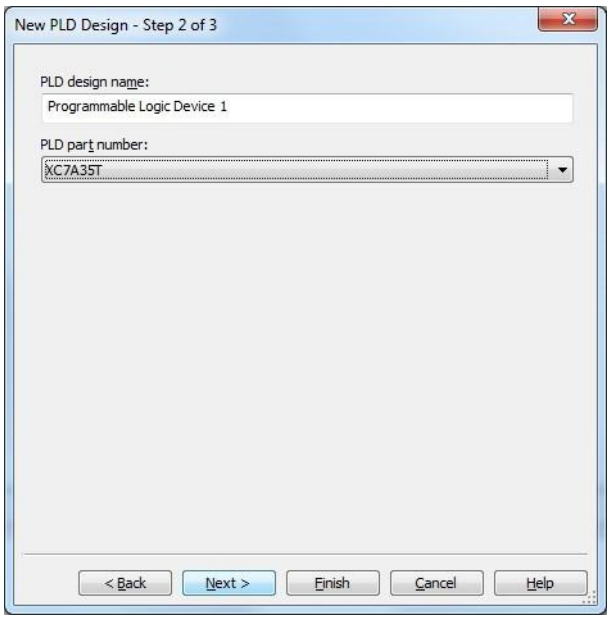
You should see a pop-up window like the one shown on the next page.



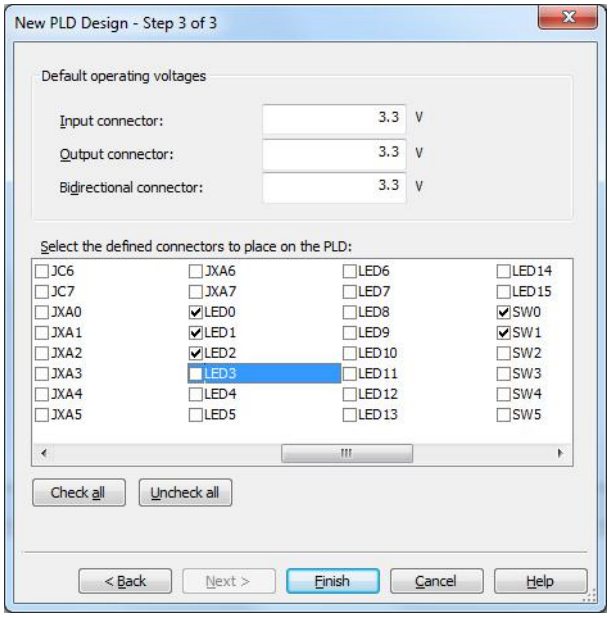
Select the **PLD design** icon as shown, so that your design is built from the correct FPGA gates and other components specific to the Xilinx FPGA on the Basys3 board (you **cannot** use the familiar TTL and CMOS logic gates and components that you may be familiar with from Year 2).

There are three more steps to the creation of the schematic, shown by the three pop-up windows below. In the first one, you **must** select Digilent Basys3 (Multisim should remember this choice later). In the second window, you are able to choose a name for the schematic. Choose one, do not use the default.

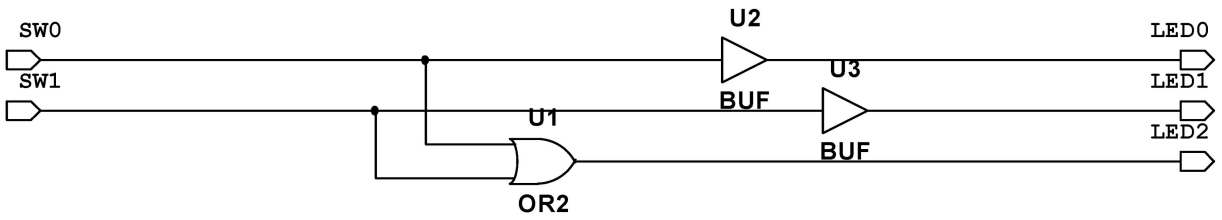




The third of these windows requires more explanation. The Multisim software knows about the Basys3 board, and it knows the names of all the I/O pins on the FPGA (known as **connectors** in Multisim). The third pop-up enables you to select the pins needed on your schematic. In this case, for the tutorial circuit, you need only the first two switch pins (SW0 and SW1) and the first three LED pins (LED0, LED1, LED2). All the other pins can be unchecked.



Edit the schematic as shown below. This is a very simple circuit, containing one gate and two buffers, driven by switches on the board. The outputs from the switches also drive two LEDS and the output of the gate drives a third LED. It is easy to check that this circuit has been correctly entered, built and downloaded to the board, by testing the four combinations of settings for the two switches and observing the three LEDs.



In my version of the circuit shown here the two-input gate is an OR, but you can choose something else such as AND/XOR/NAND/NOR.

Once the schematic is complete, it must be exported to disk in the form of VHDL files. These are text files (source code in the VHDL language). It is not too difficult to read the code and see the correspondence between features in your circuit and the code. The Multisim software inserts some components automatically, as required by the Xilinx software and the FPGA – for example, **buffers** are automatically inserted at the circuit inputs and outputs.

To export your design go to the **Transfer** menu in the menu bar, and select **Export to PLD**. In the pop-up window that appears, select **Generate and save VHDL files**. Be careful about where the files are saved, since by default they will go to **C:**. Ideally they should go in the same folder as the Multisim schematics and project files, but you could place them in a folder where your Xilinx files will be saved (see next page). It doesn't matter too much for the purpose of this tutorial, but later you should think about how to organise these files in separate folders (perhaps one folder for VHDL files).

You may notice that the Multisim software offers the option of directly downloading the design to the FPGA board. However, this relies on an older version of the Xilinx software which does not have support for the Basys3 board. Our only option is export to VHDL then Xilinx Vivado, which we will cover next.

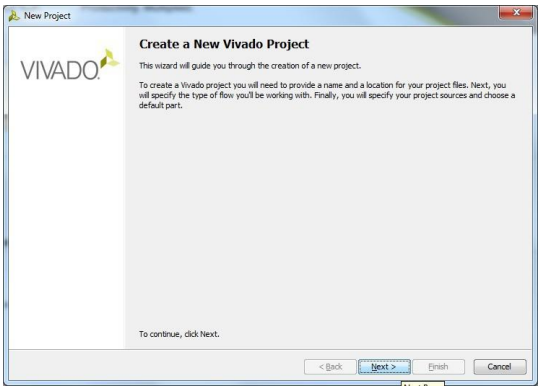
If you want to see what is in the VHDL files (there are two of them), open them with Notepad++, or wait until you have them imported into Xilinx Vivado and view them there in the Xilinx editor.

Xilinx Vivado

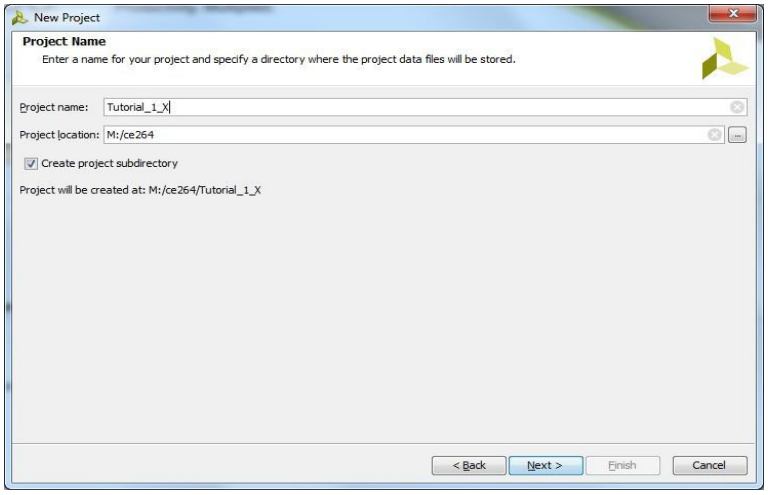
Now that you have the VHDL files created by Multisim, you will need to start the Xilinx Vivado software. From the Windows Start menu, select:

Xilinx Design Tools → Vivado 2016.2

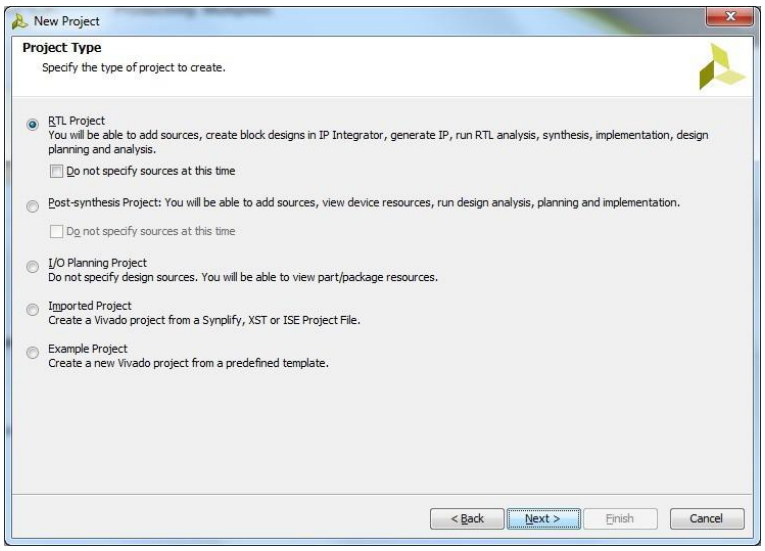
As with Multisim, you need to create a project (this will be a Xilinx project, completely independent of the Multisim project you created earlier). To create a project select the **Create New Project** icon under **Quick Start** as shown below, or select **File → New Project** from the menu bar.



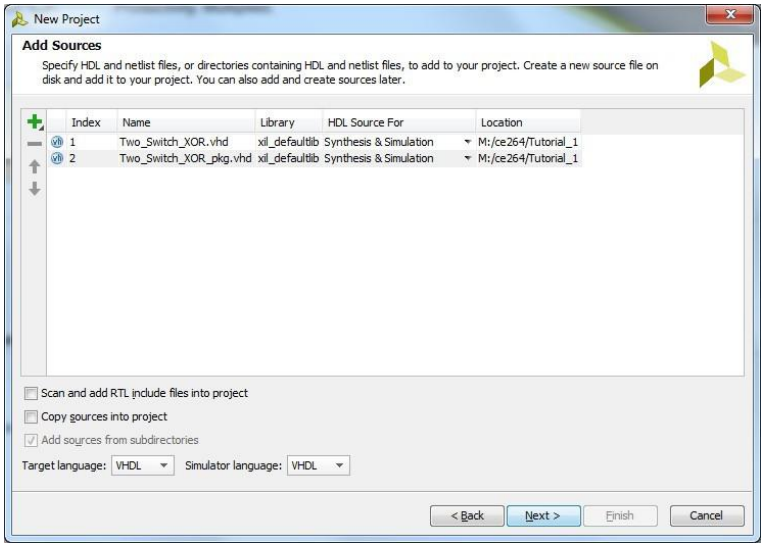
A pop-up window will appear as shown to the right. There is no real content in this window, so just click on Next and move to the following step. In the next window you must enter a name for your project and a location for the files. I recommend you to choose a name similar to but different from the name of the corresponding Multisim project. For example, use the name of the Multisim project but append an 'X'. (for Xilinx). The location should be the same as the location of the Multisim project (in my case M:\ce264).



After clicking Next, you will be asked what type of project you wish to create. Choose **RTL Project**. (RTL means ***register-transfer level***. This won't mean much to you, but it is a jargon phrase used in CAD to describe a design implemented in terms of registers and combinational logic.)



Add source



You will not be creating a design in Vivado, merely importing a design that was created in Multisim. Since the design is represented by VHDL source code files, you **Add a Source** (or sources, meaning source code files) to the project. Do not select the option to **copy** the files into the project, otherwise when you change the files in Multisim, you will need to re-copy them here into Vivado. If you don't copy them, Vivado **links** to them, and updates in Multisim will be automatically available to Vivado.

Notice that you need to select the option for **VHDL** in the **Target Language** and **Simulator Language** pull-downs at the bottom left of the window.

The following window can be skipped. 'IP' means Intellectual Property, another piece of industry jargon meaning a block design that has been implemented by someone else. IP blocks can take the form of translated logic, without source code. You don't have any of that, so just click Next.

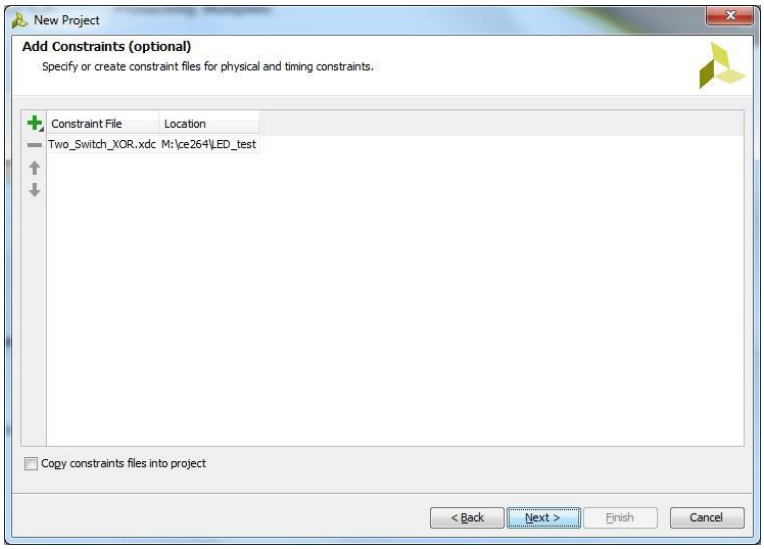
Constraints file

The schematic that you entered into Multisim specifies inputs and outputs on the FPGA by name. It does not contain information on the pin numbers which must be used. SW0, for example, means switch 0 on the Basys3 board (or more precisely, it means the pin on the FPGA which is wired to that switch on the board). In order to make your circuit work correctly on the board, we must tell the Xilinx software that the signal called SW0 must be routed to pin V17 on the FPGA (this information is given in the Basys3 reference manual, Figure 16 on page 15).

The Xilinx software requires this information to be supplied in the form of a **constraints file**. This is a text file containing statements in a simple language. Digilent provide a template file for the Basys3 board, which I have edited and placed on our Moodle site for you to download. The version of the file you need for this tutorial is called **`Two_Switch_OR.xdc`**. Download it and save it to your project folder (I'm assuming that you know how to download a text file – if you don't know **ask**). **Make sure that Windows downloads the file with exactly the filename given above.** If Windows adds **.txt** to the end of the filename, you have done something wrong, and you need to fix it, because Xilinx will not find the constraints file.

Later, you will need to edit constraint files, according to your own needs, so look at the constraints file and see what it contains. **View it using TextPad or Notepad+**, not Windows Notepad. Be careful not to alter the file (exit without saving if it says changes have been made).

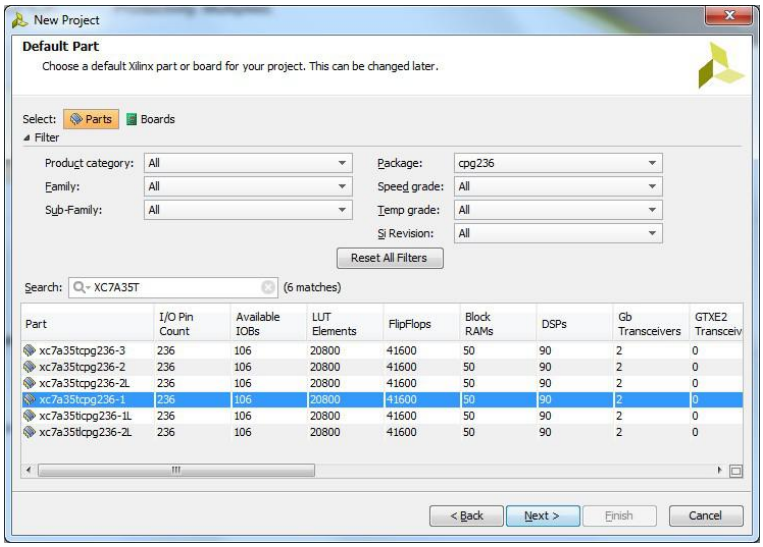
You must tell Vivado where to find the constraints file, and this is done in the next window (right), in the same way you told Vivado where to find the VHDL files. Again, do not select **copy** - you want changes to the file to be used by Vivado without having to re-copy the file into the Vivado database.



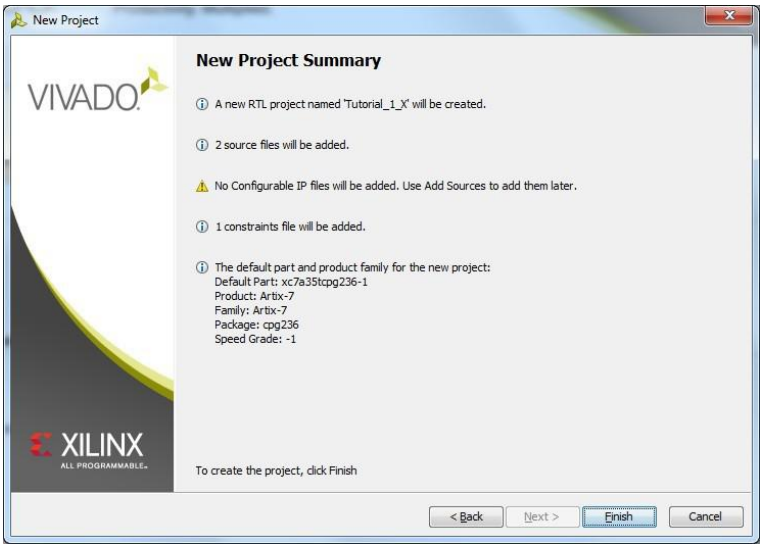
The next window is very important. Errors here will cause a failure when you try to make your design work on the FPGA board. You must select the correct 'Part' meaning an FPGA chip. At present we cannot use the board option and select Basys3 because Vivado doesn't contain this as an option (a later update may do this, but we have to use what is available). The correct FPGA part number is available in the Basys3 documentation. It is:

XC7A35T-1CPG236C.

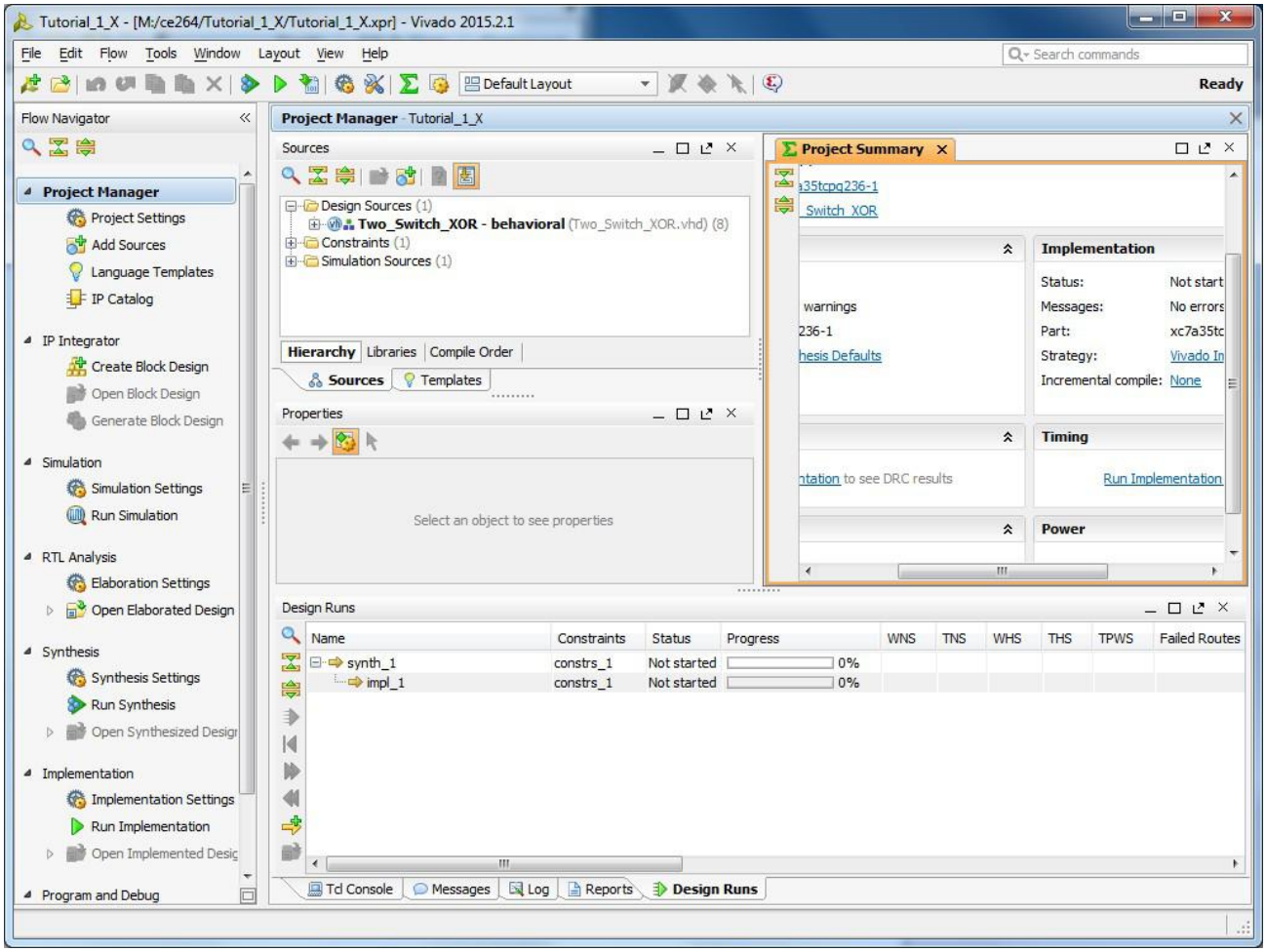
This appears in the Vivado window as **xc7a35tcpg236-1**.



Finally, the next click takes you to a confirmation window so you can check that everything is correct before finishing, at which point Vivado creates the project files and takes you into the main Vivado window.



The main window should now appear, looking like this:



This looks complicated **and it is**. Remember that Vivado is a professional-level CAD product. It is used in industry for very large and complex designs. However, you need to use only a small subset of its functionality. Before we see what to do, it is helpful to understand what the steps are.

Synthesise/Implement/Produce bit file

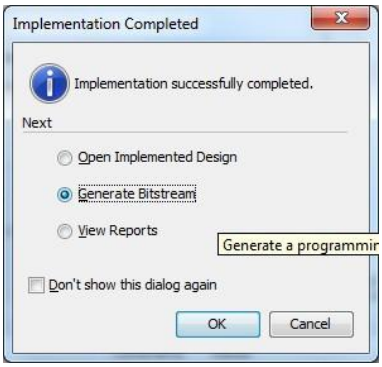
There are three stages to the conversion of VHDL code into a working design in the FPGA:

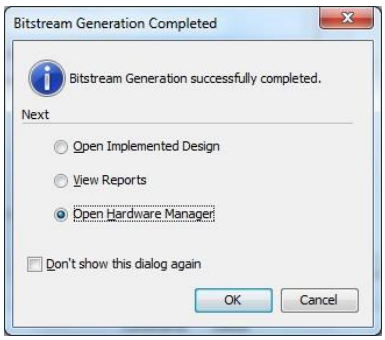
- Logic synthesis – the VHDL code is compiled and translated into a circuit expressed in primitive components that are present on the FPGA.
- Implementation – the components in the translated circuit and their interconnections, are placed on the FPGA and routed. Placement means selecting locations for each component (there are thousands of flip-flops on the FPGA, and each flip-flop in your design has to be allocated to a specific flip-flop on the chip, for example). Routeing means finding a path for each logic signal, using routeing resources (wires, buffers, switches) on the chip.
- Produce bit file – the implemented design must be translated into a large file of bits that will be downloaded to the FPGA board. These bits will configure the FPGA so that its behaviour matches what you designed.

These three steps are selected one by one from the menu on the left side of the Vivado main window under the heading 'Project Manager'. Each step produces diagnostic output. There may be errors and warnings produced. By now you should understand what these two terms mean (they are used in many contexts in software and programming), but it may be you have not yet learned this, so here is an explanation:

- Errors are **serious** and will result in the process stopping without completion. If an error occurs you must read the error message, work out what is wrong, and **fix it**. You may need help with this, but as you learn more, you will become able to work out for yourself what is wrong, and fix it without help.
- Warnings are **less serious**. Mostly they can be ignored, but sometimes they will alert you to something that should be fixed. Your circuit may work despite the warnings, but it may work better if you fix the problem that is causing the warning.

To run the synthesis step, all you have to do is click the **Run Synthesis** button. This takes a little time, and when it is done (without errors), the following pop-up will appear. Click on 'OK' and it will run the next process, implementation, at the end of which the next pop-up will appear.





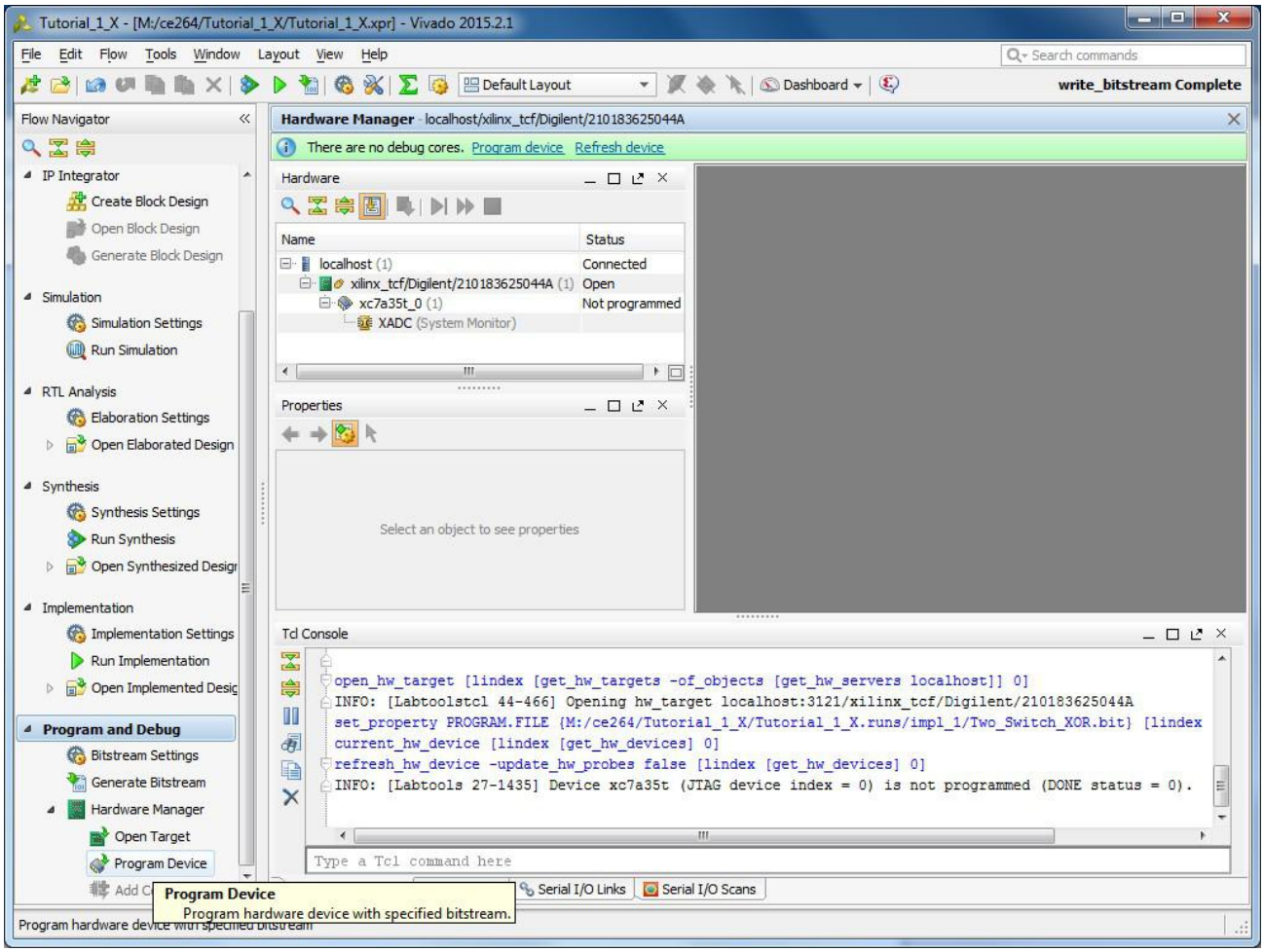
Download and test

If you have successfully created a bit file for your design, you are ready to test it on the FPGA. Xilinx Vivado can download the bit file for you. Make sure the power switch on your Basys3 board is in the OFF position, and connect your Basys3 board to a USB port on the computer. Switch on the Basys3 board, and the power-on LED should illuminate.

Now start the Vivado Hardware Manager. The button to do this is at the bottom of the Project

Manager pane, below Synthesis and Implementation. Once the hardware manager is open, the window will appear as in the screenshot below. Select the option to Program Device and your bit file should be downloaded to the board, assuming all is correct with the USB connection, and the hardware manager has connected to the board (this will be apparent from the main window – make sure it has detected the board, and if not, try auto-detect).

At the end of the download, the circuit will start to function. If you entered an OR gate, as I did, try setting SW0 and SW1 to the ON position, and LEDs 0 and 1 should illuminate. Now try the four possible combinations of positions for SW0/1 in turn and note down the results. Are these results correct according to the truth table for an OR gate, and the circuit schematic as entered? If so, well done. If not, work out what is wrong and fix it until the circuit does behave correctly. If you entered a different type of gate, check the truth table for that gate type (all input combinations).



In case of problems

If you have problems with the download, you may want to check that the Basys3 board and your USB cable are working correctly. The Basys3 board has a test feature which is documented on page 19 of the reference manual (Section 10 Built-In Self-Test). Follow the instructions there to verify that the board is working correctly (note that you can test individual switches and LEDs when this test feature is activated).

Design iteration

The above outline describes the process that you have to go through the first time you work on a design/project. If you set things up as suggested above, the process is considerably simpler after you have made a change to your design, since you don't need to set up the projects, the schematic names etc. In Multisim, all you should need to do is edit your schematic(s), save it/them to disk, and then re-export the design as VHDL files (overwriting the previous files). In Vivado, all that should be necessary is to re-run the synthesis/implementation/bitstream generation, and this can be done with a **single click** – the Generate Bitstream button in the Flow Navigator menu down the left-hand side of the Vivado window (see above, second entry under Program and Debug) will automatically re-run the synthesis step if it detects that the VHDL files have changed, and it then proceeds through to the bitstream automatically, provided that no errors are encountered.

Final remarks

The above is only an outline. You may need to vary some details as you progress during the term. Refer back to this document as needed to remind yourself of the steps, but also make notes in your laboratory notebook, so that you can reproduce these steps easily. There will be a laboratory competence test later in the term to test whether you have learned how to enter a schematic accurately and translate it into a working design on the FPGA. The steps required in the competence test will be very similar to those in this tutorial, **but with a somewhat larger circuit than just one gate**. If you have not completed or properly understood this tutorial, you will not be able to complete the competence test. However, if you have understood the tutorial a mark of 100% is possible in the competence test if you complete it correctly in the time allowed.

Dr Liang Hu
CE264 Module Supervisor
Version 1
8 February 2020